Security Suggestions for the Cashflow App and Its Flows

Project: Cashflow application, ADT Dental Clinic
Date: November 5, 2026
Status: Suggestions only. Nothing here has been built; the password-change and reset work is already done and is listed first for context.
How to read it: each item says what to do, why it matters for a clinic that holds patient and payment records, roughly how long it takes, and what to do first. Section A should be done before real patient use grows; B soon after; C when the clinic is ready.

---

1. Where the app stands

Area	State
Sign-in	Passwords hashed with bcrypt; 5 wrong tries lock an account for 15 minutes; one internet address locked after 20; an unknown e-mail takes as long to refuse as a wrong password.
Sessions	30-minute idle limit, 12-hour total limit, and now ended at once when a password changes or an account is removed.
Passwords	New: temporary passwords made by the system, forced change at first sign-in, 10-character minimum, common-password list, sign-out of other devices after every change.
Roles	Superuser (one person), admin, dentist, staff; a role change now applies on the next click.
Reset	New: forgot-password requests handled by admins, with the superuser covering admin accounts.
Pages	Security headers and content policy set; search engines told to ignore the app; health check and backup script exist.
Records	An audit trail records visit, payment, expense, patient, password and user-creation events.

A. Do these first

#	Suggestion	Why	Effort
A1	**Rotate the secrets**: the Neon database password and `AUTH_SECRET`; then store them only in the hosting provider's settings.	They were seen in files and chats during development. `AUTH_SECRET` now also protects the sign-in renewal after a password change.	1 hour
A2	**Upgrade Next.js** to the latest patched release (16.4.3 or newer) and keep it current.	Published fixes for the framework itself; this is the one dependency every request passes through.	0.5 day
A3	**Retire the shared test account and every password that has been typed in a chat or document** (`superuser@admin.com`, `admin@admin.com`, `staff1/2@dent.com`). Make a real person the superuser (decision D9), then switch on the "everyone changes their password" line at the end of the password SQL file.	Anyone who has seen those passwords can still sign in. The superuser account can approve every reset, so it must belong to a person, not to a test login.	1 hour

#	Suggestion	Why	Effort
A4	**Keep at least two admins plus the superuser**, and keep the break-glass command (`yarn db:reset-password`) on one trusted computer only.	One lost password should never block the clinic, and the command must not be easy to reach.	30 min
A5	**Finish the permission matrix** from the permissions architecture: one place that decides what each role may do, used by every page, action and API route, with the coverage check in the build.	Today each action has its own check. A missing check is invisible until someone finds it. The new `auth()` guard already gives the single entry point it needs.	3 to 4 days
A6	**Audit the role changes, branch changes and user removals.**	Creating users and resetting passwords are logged now; changing a role, moving a branch and deleting a user are not. These are exactly the events to check after an incident.	0.5 day
A7	**Test a restore of the database backup** and write down how long it takes.	A backup nobody has restored is a hope, not a backup. Patient and payment history cannot be re-typed.	2 hours
A8	**Protect spreadsheet downloads from formula injection.** In CSV and Excel exports (commission report, transactions), prefix any cell that starts with `=`, `+`, `-` or `@` so it is read as text.	A patient or procedure name typed as a formula can run in the clinic's Excel when someone opens the export.	2 hours

---

B. Do these soon

#	Suggestion	Why	Effort
B1	**Ask for the password again before dangerous actions**: deleting a user, deleting visits in bulk, resetting a password, exporting everything.	A walked-away-from signed-in screen (or a stolen session) should not be enough to do the most damaging things.	1 day
B2	**Two-step sign-in (an authenticator app) for the superuser and admins.**	A guessed or phished admin password would otherwise open every record. Two steps are the single biggest upgrade for the accounts that matter most.	3 days
B3	**A security page for admins**: sign-ins (including wrong ones and locks), resets, role changes, and a "sign out everywhere" button on Profile.	The data is already recorded (`login_attempts`, audit trail); there is no screen to read it quickly.	1.5 days
B4	**Alerts**: e-mail or chat message to the superuser when an admin or superuser signs in from a new browser, when an account is locked, and when a reset is done.	Reset requests wait in the menu today. An alert makes them visible within minutes and shows odd activity early. Needs a mail or chat service.	1 day (plus a service)
B5	**Check new passwords against known breaches** (the free "k-anonymity" service from Have I Been Pwned: only the first 5 characters of a hash leave the server).	Catches passwords that are long but already leaked.	0.5 day
B6	**Switch off people instead of deleting them** (an "active" flag), and disable accounts with no sign-in for 90 days.	Deleting a person breaks their history; a switch keeps records and still stops access. An offboarding checklist (remove access the day someone leaves) fits here.	1 day
B7	**Move the temporary password off chat.** Hand it over in person, by phone, or on a printed card;	Chat history keeps the password for its 3 days of life and often much longer.	process, or 2 days for links

#	Suggestion	Why	Effort
	later, a single-use link that expires in 30 minutes once an e-mail service exists.		
B8	**Tighten admins' reach**: let only the superuser create or demote admins and delete users who are admins.	Admins can currently make new admins. With the superuser role in place this is a small, clear rule.	0.5 day
B9	**Patient file uploads (LOA/LOE, attachments)**: store them privately, serve through short-lived links, check type and size, and scan if the clinic sends files from outside.	These are the most sensitive files in the app.	1 to 2 days
B10	**A pipeline that runs on every change**: type check, lint, the password and permission tests, dependency audit, and a build.	Catches mistakes and vulnerable packages before they reach the clinic.	1 day
B11	**Monitoring**: uptime check on `/api/health`, error alerts (for example Sentry), and a weekly look at lock-outs.	You learn about a problem before the clinic does.	0.5 day

---

C. When the clinic is ready

#	Suggestion	Why
C1	**Single sign-in with the clinic's Google or Microsoft accounts**, with the two-step rules those already provide.	One fewer password to manage, and offboarding becomes one switch.
C2	**Tighter content policy** (nonces instead of inline scripts) and a Permissions-Policy header.	Shrinks what an injected script could do if one were ever found.
C3	**Separate database roles**: one for the app, a read-only one for reports and backups, restricted by network address where the hosting allows.	A leak of one credential then cannot change or erase data.
C4	**Record who looks at which patient** (read access, not only changes) and review it monthly.	Expected under data-privacy rules where patient records are involved.
C5	**Data-privacy paperwork**: retention schedule, a named person responsible for privacy, a written breach procedure, and a review of the draft privacy pages by a lawyer.	Required work under the Data Privacy Act (RA 10173); the app's pages are drafts and checklists, not legal advice.
C6	**Move to a stronger password hash** (Argon2id) and rehash on sign-in.	Bcrypt is acceptable today; this is a long-term hardening step.
C7	**Rate-limit at the edge** (the hosting provider's firewall) in front of the sign-in and forgot-password pages.	Stops floods before they reach the database.
C8	**Yearly security review** and a small test of the reset process (a pretend request, checked end to end).	Processes drift; testing them keeps them honest.

D. Flow-by-flow notes

First sign-in (new account). Strong already. Add B7 (hand-over without chat) and B6 (switch off, not delete). Consider shortening the temporary password life to 24 hours for accounts that are created in person.

Forgot password (admin handled). The weakness is a person, not the code: an admin can be talked into giving a password to someone pretending to be a colleague. Keep the rule "check in person or call back a known number", log every reset (done), and alert the superuser (B4). Admin accounts need the superuser, which already adds a second person.

Normal sign-in. Add two-step sign-in for admins (B2) and the breach check (B5). The lock is on e-mail and address; add the edge rate-limit (C7) so an attacker cannot cause load.

Voluntary password change. Done, with the current password required, a lock on wrong attempts, and sign-out elsewhere. Add B1 for other sensitive actions.

Sessions. Idle and total limits are good. Add "sign out everywhere" and a visible "last signed in" line (B3).

Imports, exports and attachments. Add A8 and B9. Limit file size and number of rows per import; refuse unexpected file types.

Reports and money screens. Today the permission checks are per page. A5 makes them one table, so a new report cannot forget its check.

Backups and recovery. A7 first. Then keep one copy outside the hosting account and test yearly.

E. Suggested order

1. **This week:** A1, A3, A4, A7, A8 (all small).
2. **Next two weeks:** A2, A6, B8, B7, B11.
3. **Next month:** A5, B1, B3, B10.
4. **After that:** B2, B4, B5, B6, B9, then section C as the clinic grows.